Tarea 2 Redis y HBase

Del stream a la historia
Nicolas Galaz, Martin Yañez y Eduardo Roa
Procesamiento Masivo de Datos · ELE051-B / EIN102B · USM 2026-2 · Paralelo 701
Profesor Diego Ramírez · 1 de noviembre de 2026

Construimos un pipeline que separa el estado actual de la historia. Redis mantiene el tablero y entrega los eventos a dos grupos independientes. HBase conserva las mediciones y permite consultar rangos. Nuestra decisión es elegir el motor según la pregunta y comprobar el costo con consultas reales.

A y B Infraestructura y verificación

Usamos Redis 7 con AOF, appendfsync everysec y volumen; HBase 2.5.15 standalone con el Dockerfile del Lab 07 y volumen propio. Ambos tienen healthcheck. El histórico se escribe directamente en dos tablas, en lotes de 1.000. La hora en vivo entra en sensores:t2 cada 25 ms. Ingesta confirma con XACK después de escribir las dos tablas; tablero actualiza Redis y confirma dentro de un script Lua atómico.

Control	Histórico	En vivo	Total
Eventos	864000	7200	871200
Repetidos exactos	17090	153	17243
Claves distintas	846906	7047	853953

La carga demoró **129,79 s**: 6.657 eventos/s y 6.525 filas distintas/s por tabla. Cada tabla terminó con 853.953 filas; cada grupo procesó 7.200 entradas y quedó con XPENDING = 0 y lag = 0. Las cuatro colisiones con igual estación y milisegundo explican la diferencia que no corresponde a repetidos exactos.

Qué va en cada motor

Dato	Motor y estructura	Por qué lo dejamos ahí
Última lectura	Redis · hash ultimo:<estación>	Acceso directo por estación para el tablero.
Ranking actual	Redis · zset ranking:flujo	Top 5 sin recorrer la historia.
Ventana de 10 minutos	Redis · zset ventana:<estación>	Puntaje por medición y recorte por rango.
Catálogo	Redis · set tipo:<tipo>	Lista de estaciones para decidir qué consultar.
Eventos en tránsito	Redis · stream sensores:t2	Dos grupos reciben todos los eventos y conservan pendientes.
Historia por estación	HBase · estación#ts	Un intervalo de una estación queda contiguo.
Historia por tipo	HBase · tipo#ts#estación	Un intervalo del tipo necesita un solo scan.

ts es el milisegundo de medición, con 13 dígitos. Lecturas tiene cuatro regiones con cortes CC-005, HUA-001 y SPP-002, cuatro estaciones en cada una; lecturas_tipo se divide por los tres tipos. Las claves no dependen de la llegada. El consumidor único compara la secuencia antes de reescribir: un replay durable se omite, y VERSIONS = 1 conserva un valor visible. No hay una transacción entre tablas; si la segunda falla, el pendiente permite completar ambas al reintentar.

P1 El estado actual desde los dos motores

En Redis leímos los 16 hashes en un pipeline y el ranking con ZREVRANGE. En HBase hicimos 16 scans inversos por prefijo, cada uno con límite 1; así no necesitamos recorrer toda la historia para encontrar la medición más reciente. Los resultados de ambos motores coincidieron.

Estación	Valor	Medición	Estación	Valor	Medición
CC-001	33.2	00:59:58.000	HUA-001	90.1	00:59:51.000
CC-002	62.7	00:59:54.000	HUA-002	97.1	00:59:59.000
CC-003	42.6	00:59:57.500	PEN-001	68.6	00:59:47.000
CC-004	73.4	00:59:53.000	SPP-001	66.0	00:59:56.000
CC-005	19.1	00:59:52.500	SPP-002	98.0	00:59:59.500
CHI-001	67.8	00:59:56.500	TAL-001	41.0	00:59:40.500
COR-001	19.4	00:59:43.500	TAL-002	60.0	00:59:50.000
COR-002	62.8	00:59:55.500	TAL-003	45.0	00:59:58.500

Mediciones del 07-nov, hora de Chile UTC-03. Los valores tienen la unidad del sensor: flujo vehicular, ocupación o PM2.5, según el catálogo.

Top 5 de flujo SPP-002: 98,0 · HUA-002: 97,1 · HUA-001: 90,1 · PEN-001: 68,6 · CC-002: 62,7.

Redis demoró 3,10 ms y HBase 322,03 ms. Para un dashboard elegimos Redis porque el estado ya está calculado y se consulta directamente.

TAL-001 cambia con el reloj elegido. Por medición: 41.0 a las 00:59:40.500; por llegada: 48.9 medido a las 00:58:48.335, secuencia 871166. La última llegada es tardía; por eso no reemplaza el hash ni el ranking.

P2 Promedio en los últimos 10 minutos

Anclamos la ventana a 2026-11-07T00:59:59.500-03:00. Consultamos ZRANGEBYSCORE entre ancla menos 600.000 ms y ancla, incluyendo ambos extremos; el promedio se calcula con los valores leídos de ventana:*.

Estación	Lecturas	Promedio	Memoria ventana B
CC-001	75	20,89	2616
CC-002	68	58,88	2104
CC-005	70	18,67	2616
HUA-001	62	92,08	2112
HUA-002	67	76,50	2112
PEN-001	86	87,34	3136
SPP-002	75	93,13	2624
TAL-001	74	34,45	2624

La consulta demoró 21,02 ms. El score permite ubicar y borrar por tiempo de medición, aunque la llegada esté desordenada. Recortamos todas las estaciones con el watermark global: una estación sin nuevas lecturas tampoco conserva registros vencidos. El miembro combina timestamp y valor; los reenvíos no suman muestras y una colisión se resuelve por secuencia.

MEMORY USAGE de los zsets: 19,944 B con recorte y 368,344 B guardando la hora completa. Los hashes auxiliares de secuencia ocupan 10,496 B adicionales. Sin recorte, la memoria crece con toda la historia recibida.

P3 Lecturas de CC 001 el 4 de noviembre

Consultamos [08:00, 10:00) con row_start = CC-001#inicio y row_stop = CC-001#fin. Comparamos ese scan con recorrer lecturas completa y filtrar en el cliente; ambos dieron el mismo resultado.

Cuarto de hora	Lecturas	Promedio
08:00	112	15,61
08:15	125	25,67
08:30	97	17,13
08:45	104	25,87
09:00	116	24,40
09:15	91	18,10
09:30	128	19,24
09:45	118	17,49

Hay **891 lecturas**. El rango tomó 39,95 ms y el recorrido completo 23.942,17 ms, 599,4 veces más. Con cinco años, el recorrido completo debe examinar mucha más historia. El intervalo consultado conserva su tamaño; igual dependerá de los índices de bloques, HFiles, cachés y distribución de regiones. No extrapolamos el factor medido como una constante.

P4 PM2 5 del 5 de noviembre entre 07 y 09 horas

Estación	Lecturas	Promedio	Máximo	Primera hora del máximo
CC-004	877	68,09	80,0	07:22:15.000
CHI-001	904	62,18	80,0	07:07:14.500
COR-001	890	22,78	52,3	08:18:32.500
TAL-003	895	48,14	76,5	07:27:43.500

Con el catálogo tipo:calidad_aire y un rango por estación demoró 188,48 ms; con un scan en lecturas_tipo, 109,37 ms. Los promedios, máximos y primeras ocurrencias coincidieron.

La segunda tabla requiere hasta 871.200 escrituras lógicas extra, antes de consolidar repetidos en cada lote. Después de flush, lecturas ocupó 245,47 MiB y lecturas_tipo 317,96 MiB en sus directorios de tabla, sin sumar WAL ni snapshots. Si falla la segunda escritura, no confirmamos el stream; el reintento consulta la secuencia de cada tabla y completa lo que falta. No mantendríamos ese índice si casi todas las consultas fueran por estación o si el costo de escritura y disco pesara más que la mejora de las pocas consultas por tipo.

Tabla de tiempos de las consultas

Pregunta	Diseño 1 ms	Diseño 2 ms	Diseño 3 ms
P1	Redis 3,10	HBase 322,03	-
P3	Rango 39,95	Completa 23.942,17	-
P4	Estaciones 188,48	Tipo 109,37	-
P5	Estación 279,23	Tiempo 54,41	Salt 74,25

Valores en milisegundos. Cada cifra es la mediana de cinco corridas posteriores a una de calentamiento. El equipo y los límites de esta comparación se indican en la página siguiente.

P5 Hotspotting y salting

Escribimos la misma hora en tres tablas nuevas. Estación primero tiene cuatro regiones de cuatro estaciones; tiempo primero se divide por los días 03, 05 y 07-nov; salting usa SHA-256(estación) mod 4 y cortes 1#, 2# y 3#. Usamos un hash estable para que la región no cambie al reiniciar Python. Esperamos 12 s antes y después de escribir y restamos los contadores JMX.

Diseño	Región 1	Región 2	Región 3	Región 4
estacion	1761	1708	1732	1847
ts	0	0	5	7043
salt	3020	1314	1786	928

Las columnas siguen el orden de las claves de inicio. JMX cuenta las mutaciones enviadas, no los 7.200 eventos de entrada: consolidamos claves repetidas dentro de cada lote. En tiempo primero también pueden caer lecturas tardías en el día anterior; el cuadro muestra la distribución real.

p5_estacion: 1180 lecturas entre [00:10, 00:20) del 07-nov; 279,23 ms.

p5_ts: 1180 lecturas entre [00:10, 00:20) del 07-nov; 54,41 ms.

p5_salt: 1180 lecturas entre [00:10, 00:20) del 07-nov; 74,25 ms.

Los tres resultados completos coincidieron y guardamos su hash de control. Estación primero necesita 16 rangos, tiempo primero uno y salting cuatro. Para un servicio que recibe continuamente datos de muchas estaciones elegiríamos estación primero si domina la consulta individual. Si dominan los intervalos globales y existen varios RegionServers, consideraríamos salting para repartir la entrada. Tiempo primero es cómodo para consultar, pero concentra las escrituras recientes. El hash por estación reparte según el catálogo: cuatro salts no garantizan cuatro cargas iguales.

P6 Nuestra pregunta

¿Qué comuna presenta ahora mayor diferencia de flujo entre sus estaciones y en qué intervalo de cinco minutos tuvo su mayor flujo agregado entre 00:00 y 01:00 del día anterior? Sólo comparamos comunas con al menos dos estaciones de flujo. Redis selecciona la comuna con los hashes actuales y el catálogo; HBase busca el intervalo histórico. Por intervalo sumamos el promedio de cada estación y exigimos que todas tengan muestras; los empates se resuelven por la primera hora.

Ganó **CC**, con una diferencia actual de 43,6. El 06-nov su pico agregado fue [00:45, 00:50), con suma de promedios 131,46. La consulta conjunta tomó 88,89 ms. Necesitamos ambos motores: el estado actual decide qué comuna revisar y la historia decide cuándo ocurrió su pico.

Hicimos una corrida de calentamiento y después cinco corridas medidas; informamos la mediana de esas cinco. Los JSON guardan las seis duraciones. Las consultas incluyen lectura, transferencia y agregación, pero dejan la conexión ya abierta cuando es posible.

Equipo: AMD Ryzen 5 7520U with Radeon Graphics; RAM física 7.28 GiB; Windows-10-10.0.26200-SP0; Python 3.11.9; Docker 29.7.2 con 8 CPU lógicas y 3.47 GiB de RAM. Redis 7.4.2 y HBase 2.5.15 standalone. Las regiones comparten un único proceso y disco: estos tiempos no miden escalamiento de un cluster.

D1 El consumidor muere antes del XACK

Matamos ingesta-que-muere con os._exit(1), después del put de las dos tablas y antes del XACK. El proceso terminó con código 1; quedaron 200 entradas en XPENDING, asignadas a ese consumidor. Ingesta-rescate las recuperó con XAUTOCLAIM después de 5.000 ms de inactividad y procesó el resto del stream.

Muestra: CC-003#1794020400000. Antes: [{'valor': '53.9', 'version_timestamp': 1793570302558}]. Después: [{'valor': '53.9', 'version_timestamp': 1793570302558}].

El reintento encuentra la misma clave y secuencia, por lo que omite el put ya durable; la muestra mantiene el valor y su timestamp de versión. VERSIONS = 1 limita las versiones visibles. Las filas finales fueron 853.953 en cada tabla y XPENDING volvió a cero. La clave idempotente evita duplicar filas; el orden put y después XACK evita confirmar algo que todavía no fue escrito.

D2 Reiniciamos Redis en un punto controlado

Repetimos docker compose restart redis con AOF y con appendonly no más save vacío. En cada caso el pipeline quedó detenido con 3.000 eventos producidos: 2.800 confirmados en HBase, 100 entregados a ingesta sin put ni XACK y 100 todavía no entregados. El tablero ya había procesado los 3.000. Usamos tablas de experimento separadas para que la carga final no oculte pérdidas.

Estado	Antes	Después con AOF	Después sin persistencia
Stream entradas	3000	3000	0
Hashes ultimo	16	16	0
Ventanas estaciones	8	8	0
Lecturas en ventanas	572	572	0
Grupos	2	2	0
XPENDING ingesta	100	100	sin grupo

Con AOF sobrevivieron también las identidades de consumidores, los pendientes y el contenido completo del stream y del tablero; comparamos hashes de contenido. Sin persistencia se perdió todo el estado de Redis. Cada tabla de prueba de HBase conservó 2.747 filas en ambos casos. El histórico y lo ya confirmado no dependen de reiniciar Redis.

En ese punto hay **200 eventos sin confirmación durable**: 100 s del reloj simulado o 5 s de producción acelerada. Es una ventana de exposición, no necesariamente 200 claves nuevas, porque también hay repetidos. El lote escrito sin XACK de D1 ya estaba a salvo y sólo requería reproceso. En un sistema real aceptaríamos como máximo alrededor de un segundo ante un corte brusco, por eso elegimos AOF everysec; si no aceptáramos esa ventana usaríamos always y mediríamos su costo. Este reinicio fue ordenado y no demuestra tolerancia a cortes de energía ni una cota absoluta de fsync.

Límites y qué cambiaríamos

MAXLEN aproximado de 20.000 conserva la hora completa y equivale a unos 500 s de producción a 40 eventos/s. Recortar una entrada no leída puede perderla aunque exista el grupo; en producción vigilaríamos lag y aplicaríamos backpressure antes de alcanzar ese límite. HBase standalone y su WAL sobre disco local no tienen las garantías de HDFS. La URL principal de descarga devolvió 404 y el archivo alternativo resultó lento; usamos un espejo con checksum SHA-512 oficial. Mantendríamos un escritor de ingesta; para paralelizarlo agregaríamos control atómico de secuencia por fila.

Fuentes: pauta Tarea 2 Redis + HBase; laboratorios 06 y 07, github.com/diegoalrv/ein102b; Redis Docs (Streams, XAUTOCLAIM y persistencia); Apache HBase Reference Guide; HappyBase API. Código, enlaces exactos, muestras y tiempos crudos están en el repositorio.